Employee Management System

1, Project Overview

- **Title of the Project:** Employee Management System.
- **Package Name:** MiniPorjectOne.
- **Purpose**: Employee Management System is a web-based application that enables seamless employee data management within an organization.
- **Architecture:** MVC design pattern.
- **Core Functionality**: The system allows to perform CRUD operations on employee records
- **Technologies Used:**
  - Front-end - HTML, CSS and JavaScript.
  - Back-end - Spring Boot(Java).
  - Database - MySQL.
  - API Documantation: Postman.
- **Spring Dependencies(Backend/Java):**
  - Spring Web.
  - Spring Data JPA.
  - MySQL Driver.
  - Spring Boot DevTools
  - Lombok

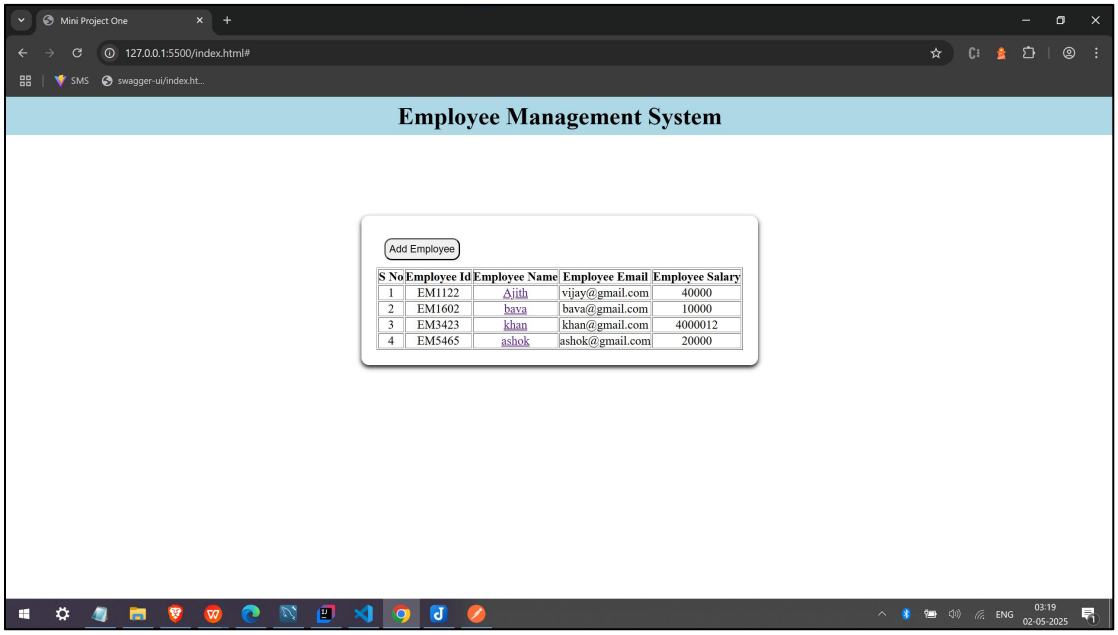
2, Database Structure

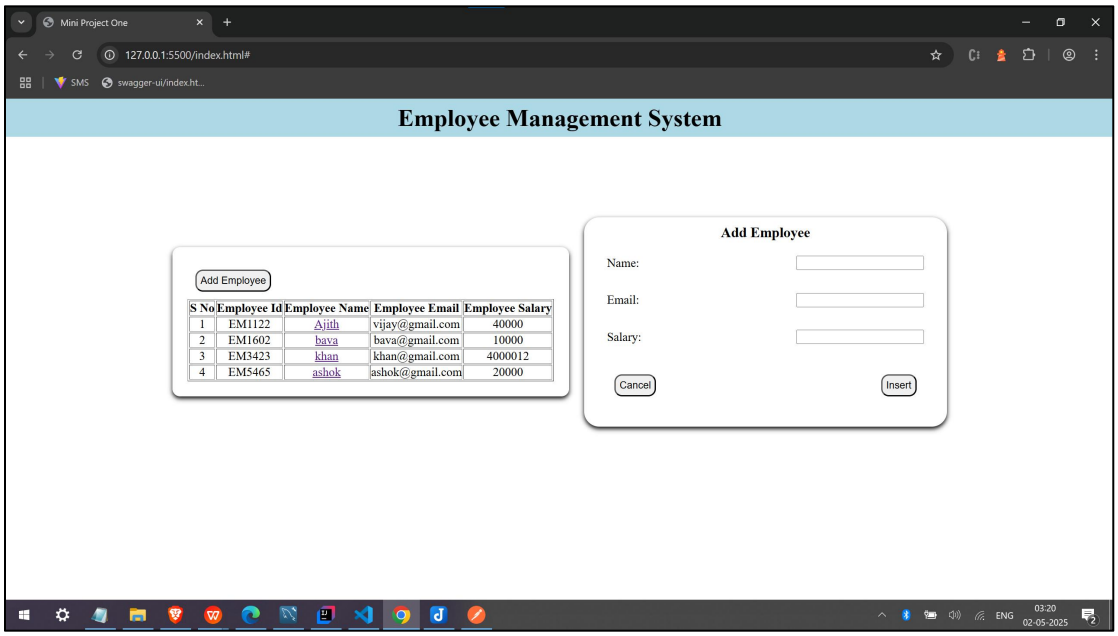
Column	Type	Description
employee_id	String(PK)	Employee unique identifier
employee_name	String	Name of the employee
employee_email	String(Unique)	Employee's email address
employee_salary	Double	Employee's salary

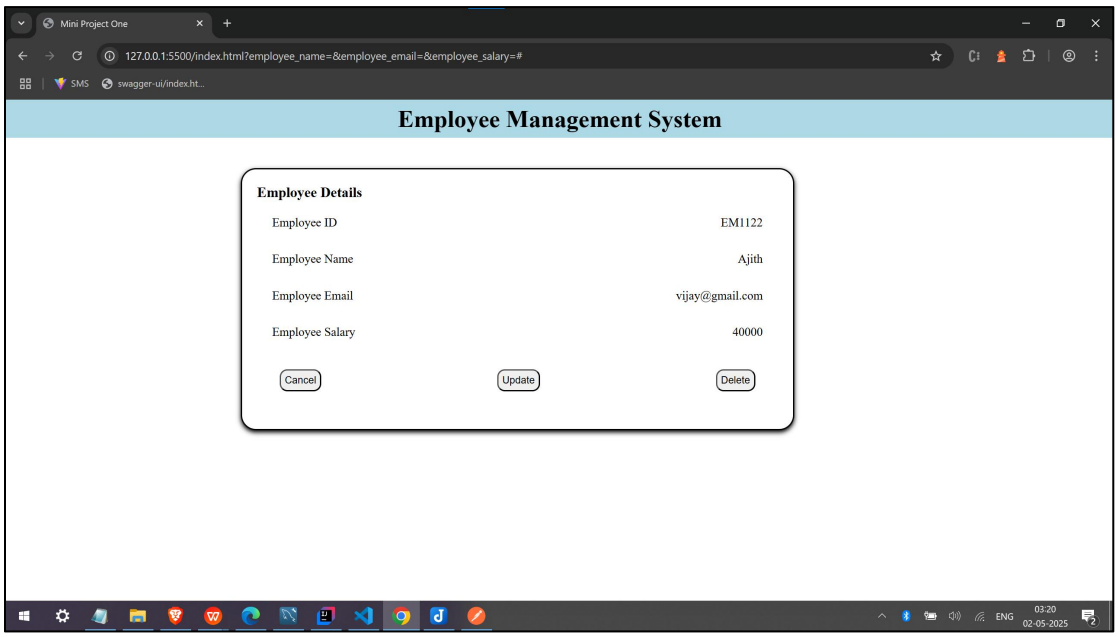
3, Frontend

- HTML for Structure
  - Form for add employee
  - Table for show list of employees
  - Details panel for read, update and delete employee's details
- CSS for Styling
  - Styled the frontend for responsiveness and a professional
- JavaScript for Interaction
  - Implemented client-side validations.
  - Used fetch API for calling backend endpoints.

Frontend Screenshots:







4, Backend (Spring Boot)

- **EmployeeController/ Controller Layer:**
  - **GET/employee** - Fetch Employees list (employee_id, employee_name, employee_email and employee_salary).
  - **GET/employee/{employee_id}** - Fetch employee based on the employee id.
  - **GET/employee/search/{employee_email}** - Fetch employee based on the employee email.
  - **POST/employee/add** - Add a new employee.
  - **PUT/update/{employee_id}** - Update employee information based on employee id.
  - **DELETE/delete/{employee_id}** - Delete employee information based on employee id.

- **EmployeeRepository / Database Interaction:**
  - Used JPA (Java Persistence API) for connect spring into database.

- **EmployeeService / Service Layer:**
  - Encapsulated business logic in service classes

- **Global Exception Handling:**
  - Used @ControllerAdvice to handle exceptions like ResourceNotFoundException.

- **Application.properties**

```
#Custom port
server.port=3000

#mysql config
spring.datasource.url=jdbc:mysql://localhost:3310/miniprojectonedb
spring.datasource.username=root
spring.datasource.password=root
string.datasource.driver.class-name=com.mysql.cj.jdbc.Driver

#JPA config
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```